

Technical Challenge: End-to-End Data Engineering & Infrastructure

1. Objective

The goal of this challenge is to build a robust, reproducible data ecosystem. You will act as both a **Backend Engineer** (building a source API) and a **Data Engineer** (building the ingestion and validation layer), while ensuring the entire stack is containerized and production-ready.

2. System Architecture

You are required to build and orchestrate three distinct components:

1. **The Source (API):** A FastAPI server providing synthetic market data.
2. **The Sink (Database):** A relational database (PostgreSQL preferred) to store processed data.
3. **The Pipeline (ETL):** A Python-based service that moves data from Source to Sink with built-in logic and quality controls.

3. Detailed Requirements

Task 1: The Mock API Server (Python/FastAPI)

Build a service that simulates a real-time financial data feed.

- **Endpoint:** GET /v1/market-data
- **Payload Structure:** Return a JSON list of objects containing:
 - **instrument_id:** (String) e.g., "AAPL", "BTC-USD".
 - **price:** (Float) Current market price.
 - **volume:** (Float) Quantity traded.
 - **timestamp:** (ISO 8601 String).
- **Chaos Engineering:** To test the resilience of your ETL, implement a "Fault Injection" flag. For **5% of requests**, the API should either return a 500 Internal Server Error or provide malformed data (e.g., a string in a numeric field).

Task 2: The ETL Pipeline & Quality Control

Develop a Python-based pipeline to process and move the data.

- **Extraction:** Efficiently poll the API. Handle potential network timeouts and the "faulty" responses defined in Task 1.
- **Transformation & Logic:**
 - **VWAP Calculation:** Calculate the Volume Weighted Average Price per instrument_id.
 - **Outlier Detection:** Flag any record where the price deviates by more than 15% from the average price of that instrument in the current batch.
- **Quality Control (QC):**
 - **Schema Validation:** Use **Pydantic** to validate incoming data types before processing.
 - **Integrity:** Ensure no duplicate timestamp + instrument_id combinations are written to the DB.
 - **Logging:** Implement structured logging to report: *Records Processed*, *Records Dropped (Validation Failure)*, and *Execution Time*.

Task 3: Infrastructure as Code (Docker)

The entire solution must be portable.

- **Containerization:** Provide a Dockerfile for the API and the ETL services.
- **Orchestration:** Provide a docker-compose.yml file to spin up the API, the PostgreSQL database, and the ETL script.
- **Secret Management:** Use a .env file for database credentials. **Do not hardcode sensitive information.**
- **Networking:** Services must communicate via the Docker internal network.

4. Conceptual / System Design (README)

In your README.md, please provide brief answers to the following:

1. **Scaling:** How would your architecture change if the data volume increased to 1 billion events per day? (e.g., Kafka, Spark, or Cloud-native tools).
2. **Monitoring:** How would you implement "Health Checks" to ensure the pipeline is running correctly in a production environment?
3. **Recovery:** If the pipeline fails midway through a 10GB batch, how do you ensure "Idempotency" (no partial or duplicate data)?

5. Submission Deliverables

1. **Codebase:** A link to a private/public repository (GitHub/Bitbucket).
2. **Infrastructure:** A working docker-compose.yml that allows us to run the entire project with one command: docker-compose up.
3. **Documentation:** A README.md with setup instructions and the system design answers.